

Arrays & Strings Assignment

Instructions: Follow the same pattern as before —

- 1. Dry run each input on paper,**
- 2. Write and test your code,**
- 3. Submit both your dry run and final working code.**

YENJOYY !!

1. Third Maximum Number

Given an integer array `nums`, return the third distinct maximum number in this array. If the third maximum does not exist, return the maximum number.

Example 1:

Input: `nums = [3,2,1]`

Output: 1

Explanation: The first distinct maximum is 3. The second distinct maximum is 2. The third distinct maximum is 1.

Example 2:

Input: `nums = [1,2]`

Output: 2

Explanation: The first distinct maximum is 2. The second distinct maximum is 1. The third distinct maximum does not exist, so the maximum (2) is returned instead.

Example 3:

Input: `nums = [2,2,3,1]`

Output: 1

Explanation: The first distinct maximum is 3. The second distinct maximum is 2 (both 2's are counted together since they have the same value). The third distinct maximum is 1.

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$

2. 4Sum

Given an array `nums` of `n` integers, return an array of all the unique quadruplets `[nums[a], nums[b], nums[c], nums[d]]` such that:

$0 \leq a, b, c, d < n$

a, b, c, and d are distinct.

$\text{nums}[a] + \text{nums}[b] + \text{nums}[c] + \text{nums}[d] == \text{target}$

You may return the answer in any order.

Example 1:

Input: $\text{nums} = [1, 0, -1, 0, -2, 2]$, $\text{target} = 0$

Output: $[[-2, -1, 1, 2], [-2, 0, 0, 2], [-1, 0, 0, 1]]$

Example 2:

Input: $\text{nums} = [2, 2, 2, 2, 2]$, $\text{target} = 8$

Output: $[[2, 2, 2, 2]]$

Constraints:

- $1 \leq \text{nums.length} \leq 200$
- $-10^9 \leq \text{nums}[i] \leq 10^9$
- $-10^9 \leq \text{target} \leq 10^9$

3. Minimum Time to Complete Trips

You are given an array `time` where `time[i]` denotes the time taken by the *i*th bus to complete one trip.

Each bus can make multiple trips successively; that is, the next trip can start immediately after completing the current trip. Also, each bus operates independently; that is, the trips of one bus do not influence the trips of any other bus.

You are also given an integer `totalTrips`, which denotes the number of trips all buses should make in total. Return the minimum time required for all buses to complete at least `totalTrips` trips.

Example 1:

Input: $\text{time} = [1, 2, 3]$, $\text{totalTrips} = 5$

Output: 3

Explanation:

- At time $t = 1$, the number of trips completed by each bus are $[1, 0, 0]$. The total number of trips completed is $1 + 0 + 0 = 1$.
- At time $t = 2$, the number of trips completed by each bus are $[2, 1, 0]$. The total number of trips completed is $2 + 1 + 0 = 3$.
- At time $t = 3$, the number of trips completed by each bus are $[3, 1, 1]$. The total number of trips completed is $3 + 1 + 1 = 5$.
So the minimum time needed for all buses to complete at least 5 trips is 3.

Example 2:

Input: time = [2], totalTrips = 1

Output: 2

Explanation:

There is only one bus, and it will complete its first trip at $t = 2$.

So the minimum time needed to complete 1 trip is 2.

Constraints:

- $1 \leq \text{time.length} \leq 10^5$
- $1 \leq \text{time}[i], \text{totalTrips} \leq 10^7$

4. Array of Doubled Pairs

Given an integer array of even length arr, return true if it is possible to reorder arr such that $\text{arr}[2 * i + 1] = 2 * \text{arr}[2 * i]$ for every $0 \leq i < \text{len}(\text{arr}) / 2$, or false otherwise.

Example 1:

Input: arr = [3,1,3,6]

Output: false

Example 2:

Input: arr = [2,1,2,6]

Output: false

Example 3:

Input: arr = [4,-2,2,-4]

Output: true

Explanation: We can take two groups, [-2,-4] and [2,4] to form [-2,-4,2,4] or [2,4,-2,-4].

Constraints:

- $2 \leq \text{arr.length} \leq 3 * 10^4$
- arr.length is even.
- $-10^5 \leq \text{arr}[i] \leq 10^5$

5. Add Minimum Number of Rungs

You are given a strictly increasing integer array rungs that represents the height of rungs on a ladder. You are currently on the floor at height 0, and you want to reach the last rung.

You are also given an integer `dist`. You can only climb to the next highest rung if the distance between where you are currently at (the floor or on a rung) and the next rung is at most `dist`. You are able to insert rungs at any positive integer height if a rung is not already there.

Return the minimum number of rungs that must be added to the ladder in order for you to climb to the last rung.

Example 1:

Input: `rungs = [1,3,5,10]`, `dist = 2`

Output: 2

Explanation:

You currently cannot reach the last rung.

Add rungs at heights 7 and 8 to climb this ladder.

The ladder will now have rungs at `[1,3,5,7,8,10]`.

Example 2:

Input: `rungs = [3,6,8,10]`, `dist = 3`

Output: 0

Explanation:

This ladder can be climbed without adding additional rungs.

Example 3:

Input: `rungs = [3,4,6,7]`, `dist = 2`

Output: 1

Explanation:

You currently cannot reach the first rung from the ground.

Add a rung at height 1 to climb this ladder.

The ladder will now have rungs at `[1,3,4,6,7]`.

Constraints:

- $1 \leq \text{rungs.length} \leq 10^5$
- $1 \leq \text{rungs}[i] \leq 10^9$
- $1 \leq \text{dist} \leq 10^9$
- `rungs` is strictly increasing.

6. Median of Two Sorted Arrays

Given two sorted arrays `nums1` and `nums2` of size `m` and `n` respectively, return the median of the two sorted arrays.

The overall run time complexity should be $O(\log(m+n))$.

Example 1:

Input: `nums1 = [1,3]`, `nums2 = [2]`

Output: 2.00000

Explanation: merged array = [1,2,3] and median is 2.

Example 2:

Input: nums1 = [1,2], nums2 = [3,4]

Output: 2.50000

Explanation: merged array = [1,2,3,4] and median is $(2 + 3) / 2 = 2.5$.

Constraints:

- `nums1.length == m`
- `nums2.length == n`
- $0 \leq m \leq 1000$
- $0 \leq n \leq 1000$
- $1 \leq m + n \leq 2000$
- $-10^6 \leq \text{nums1}[i], \text{nums2}[i] \leq 10^6$

7. Minimum Elements to Add to Form a Given Sum

You are given an integer array `nums` and two integers `limit` and `goal`. The array `nums` has an interesting property that $\text{abs}(\text{nums}[i]) \leq \text{limit}$.

Return the minimum number of elements you need to add to make the sum of the array equal to `goal`. The array must maintain its property that $\text{abs}(\text{nums}[i]) \leq \text{limit}$.

Note that $\text{abs}(x)$ equals x if $x \geq 0$, and $-x$ otherwise.

Example 1:

Input: `nums = [1,-1,1]`, `limit = 3`, `goal = -4`

Output: 2

Explanation: You can add -2 and -3, then the sum of the array will be $1 - 1 + 1 - 2 - 3 = -4$.

Example 2:

Input: `nums = [1,-10,9,1]`, `limit = 100`, `goal = 0`

Output: 1

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $1 \leq \text{limit} \leq 10^6$
- $-\text{limit} \leq \text{nums}[i] \leq \text{limit}$
- $-10^9 \leq \text{goal} \leq 10^9$

8. Candy

There are n children standing in a line. Each child is assigned a rating value given in the integer array `ratings`.

You are giving candies to these children subjected to the following requirements:

- Each child must have at least one candy.
- Children with a higher rating get more candies than their neighbors.

Return the minimum number of candies you need to have to distribute the candies to the children.

Example 1:

Input: `ratings = [1,0,2]`

Output: 5

Explanation: You can allocate to the first, second and third child with 2, 1, 2 candies respectively.

Example 2:

Input: `ratings = [1,2,2]`

Output: 4

Explanation: You can allocate to the first, second and third child with 1, 2, 1 candies respectively. The third child gets 1 candy because it satisfies the above two conditions.

Constraints:

- $n == \text{ratings.length}$
- $1 \leq n \leq 2 * 10^4$
- $0 \leq \text{ratings}[i] \leq 2 * 10^4$

9. Type of Triangle

You are given a 0-indexed integer array `nums` of size 3 which can form the sides of a triangle.

A triangle is called equilateral if it has all sides of equal length.

A triangle is called isosceles if it has exactly two sides of equal length.

A triangle is called scalene if all its sides are of different lengths.

Return a string representing the type of triangle that can be formed or "none" if it cannot form a triangle.

Example 1:

Input: `nums = [3,3,3]`

Output: "equilateral"

Explanation: Since all the sides are of equal length, therefore, it will form an equilateral triangle.

Example 2:

Input: `nums = [3,4,5]`

Output: "scalene"

Explanation:

$\text{nums}[0] + \text{nums}[1] = 3 + 4 = 7$, which is greater than $\text{nums}[2] = 5$.

$\text{nums}[0] + \text{nums}[2] = 3 + 5 = 8$, which is greater than $\text{nums}[1] = 4$.

$\text{nums}[1] + \text{nums}[2] = 4 + 5 = 9$, which is greater than $\text{nums}[0] = 3$.

Since the sum of the two sides is greater than the third side for all three cases, therefore, it can form a triangle.

As all the sides are of different lengths, it will form a scalene triangle.

Constraints:

- $\text{nums.length} == 3$
- $1 \leq \text{nums}[i] \leq 100$

10. Maximum Product of Three Numbers

Given an integer array `nums`, find three numbers whose product is maximum and return the maximum product.

Example 1:

Input: `nums = [1,2,3]`

Output: 6

Example 2:

Input: `nums = [1,2,3,4]`

Output: 24

Example 3:

Input: `nums = [-1,-2,-3]`

Output: -6

Constraints:

- $3 \leq \text{nums.length} \leq 10^4$
- $-1000 \leq \text{nums}[i] \leq 1000$

11. Subarray Sum Equals K

Given an array of integers `nums` and an integer `k`, return the total number of subarrays whose sum equals to `k`.

A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: `nums = [1,1,1]`, `k = 2`

Output: 2

Example 2:

Input: `nums = [1,2,3]`, `k = 3`

Output: 2

Constraints:

- $1 \leq \text{nums.length} \leq 2 * 10^4$
- $-1000 \leq \text{nums}[i] \leq 1000$
- $-10^7 \leq k \leq 10^7$

12. Find Original Array From Doubled Array

An integer array `original` is transformed into a doubled array `changed` by appending twice the value of every element in `original`, and then randomly shuffling the resulting array.

Given an array `changed`, return `original` if `changed` is a doubled array. If `changed` is not a doubled array, return an empty array. The elements in `original` may be returned in any order.

Example 1:

Input: `changed = [1,3,4,2,6,8]`

Output: `[1,3,4]`

Explanation: One possible original array could be `[1,3,4]`:

- Twice the value of 1 is 2
 - Twice the value of 3 is 6
 - Twice the value of 4 is 8
- Other original arrays could be `[4,3,1]` or `[3,1,4]`.

Example 2:

Input: `changed = [6,3,0,1]`

Output: `[]`

Explanation: `changed` is not a doubled array.

Example 3:

Input: `changed = [1]`

Output: `[]`

Explanation: `changed` is not a doubled array.

Constraints:

- $1 \leq \text{changed.length} \leq 10^5$

- $0 \leq \text{changed}[i] \leq 10^5$

13. Check if Array is Good

You are given an integer array `nums`. We consider an array good if it is a permutation of an array `base[n]`.

`base[n] = [1, 2, ..., n - 1, n, n]` (in other words, it is an array of length $n + 1$ which contains 1 to $n - 1$ exactly once, plus two occurrences of n). For example, `base[1] = [1, 1]` and `base[3] = [1, 2, 3, 3]`.

Return true if the given array is good, otherwise return false.

Note: A permutation of integers represents an arrangement of these numbers.

Example 1:

Input: `nums = [2, 1, 3]`

Output: false

Explanation: Since the maximum element of the array is 3, the only candidate n for which this array could be a permutation of `base[n]`, is $n = 3$. However, `base[3]` has four elements but array `nums` has three. Therefore, it can not be a permutation of `base[3] = [1, 2, 3, 3]`. So the answer is false.

Example 2:

Input: `nums = [1, 3, 3, 2]`

Output: true

Explanation: Since the maximum element of the array is 3, the only candidate n for which this array could be a permutation of `base[n]`, is $n = 3$. It can be seen that `nums` is a permutation of `base[3] = [1, 2, 3, 3]` (by swapping the second and fourth elements in `nums`, we reach `base[3]`). Therefore, the answer is true.

Example 3:

Input: `nums = [1, 1]`

Output: true

Explanation: Since the maximum element of the array is 1, the only candidate n for which this array could be a permutation of `base[n]`, is $n = 1$. It can be seen that `nums` is a permutation of `base[1] = [1, 1]`. Therefore, the answer is true.

Example 4:

Input: `nums = [3, 4, 4, 1, 2, 1]`

Output: false

Explanation: Since the maximum element of the array is 4, the only candidate n for which this array could be a permutation of `base[n]`, is $n = 4$. However, `base[4]` has five elements but array `nums` has six. Therefore, it can not be a permutation of `base[4] = [1, 2, 3, 4, 4]`. So the answer is false.

Constraints:

$1 \leq \text{nums.length} \leq 100$
 $1 \leq \text{nums}[i] \leq 200$

14. Check if One String Swap Can Make Strings Equal

You are given two strings `s1` and `s2` of equal length. A string swap is an operation where you choose two indices in a string (not necessarily different) and swap the characters at these indices.

Return true if it is possible to make both strings equal by performing at most one string swap on exactly one of the strings. Otherwise, return false.

Example 1:

Input: `s1 = "bank", s2 = "kanb"`

Output: true

Explanation: For example, swap the first character with the last character of `s2` to make "bank".

Example 2:

Input: `s1 = "attack", s2 = "defend"`

Output: false

Explanation: It is impossible to make them equal with one string swap.

Example 3:

Input: `s1 = "kelb", s2 = "kelb"`

Output: true

Explanation: The two strings are already equal, so no string swap operation is required.

Constraints:

$1 \leq s1.length, s2.length \leq 100$
 $s1.length == s2.length$
`s1` and `s2` consist of only lowercase English letters.

15. Most Frequent Even Element

Given an integer array `nums`, return the most frequent even element.

If there is a tie, return the smallest one. If there is no such element, return -1.

Example 1:

Input: nums = [0,1,2,2,4,1]

Output: 2

Explanation:

The even elements are 0, 2, and 4. Of these, 2 and 4 appear the most.

We return the smallest one, which is 2.

Example 2:

Input: nums = [4,4,4,9,2,4]

Output: 4

Explanation: 4 is the even element appears the most.

Example 3:

Input: nums = [29,47,21,41,13,37,25,7]

Output: -1

Explanation: There is no even element.

Constraints:

$1 \leq \text{nums.length} \leq 2000$

$0 \leq \text{nums}[i] \leq 10^5$

16. Buddy Strings

Given two strings s and goal, return true if you can swap two letters in s so the result is equal to goal, otherwise, return false.

Swapping letters is defined as taking two indices i and j (0-indexed) such that $i \neq j$ and swapping the characters at $s[i]$ and $s[j]$.

For example, swapping at indices 0 and 2 in "abcd" results in "cbad".

Example 1:

Input: s = "ab", goal = "ba"

Output: true

Explanation: You can swap $s[0] = 'a'$ and $s[1] = 'b'$ to get "ba", which is equal to goal.

Example 2:

Input: s = "ab", goal = "ab"

Output: false

Explanation: The only letters you can swap are $s[0] = 'a'$ and $s[1] = 'b'$, which results in "ba" $\neq$ goal.

Example 3:

Input: s = "aa", goal = "aa"

Output: true

Explanation: You can swap s[0] = 'a' and s[1] = 'a' to get "aa", which is equal to goal.

Constraints:

$1 \leq \text{s.length}, \text{goal.length} \leq 2 * 10^4$

s and goal consist of lowercase letters.

17. Word Pattern

Given a pattern and a string s, find if s follows the same pattern.

Here follow means a full match, such that there is a bijection between a letter in pattern and a non-empty word in s. Specifically:

- Each letter in pattern maps to exactly one unique word in s.
- Each unique word in s maps to exactly one letter in pattern.
- No two letters map to the same word, and no two words map to the same letter.

Example 1:

Input: pattern = "abba", s = "dog cat cat dog"

Output: true

Explanation:

The bijection can be established as:

'a' maps to "dog".

'b' maps to "cat".

Example 2:

Input: pattern = "abba", s = "dog cat cat fish"

Output: false

Example 3:

Input: pattern = "aaaa", s = "dog cat cat dog"

Output: false

Constraints:

- $1 \leq \text{pattern.length} \leq 300$

- pattern contains only lower-case English letters.
- $1 \leq s.length \leq 3000$
- s contains only lowercase English letters and spaces ' '.
- s does not contain any leading or trailing spaces.
- All the words in s are separated by a single space.

18. Shortest Distance to Target String in a Circular Array

You are given a 0-indexed circular string array **words** and a string **target**. A circular array means that the array's end connects to the array's beginning.

Formally, the next element of `words[i]` is `words[(i + 1) % n]` and the previous element of `words[i]` is `words[(i - 1 + n) % n]`, where `n` is the length of `words`. Starting from `startIndex`, you can move to either the next word or the previous word with 1 step at a time.

Return the shortest distance needed to reach the string `target`. If the string `target` does not exist in `words`, return -1.

Example 1:

Input: `words = ["hello","i","am","leetcode","hello"]`, `target = "hello"`, `startIndex = 1`

Output: 1

Explanation:

We start from index 1 and can reach "hello" by

- moving 3 units to the right to reach index 4.
 - moving 2 units to the left to reach index 4.
 - moving 4 units to the right to reach index 0.
 - moving 1 unit to the left to reach index 0.
- The shortest distance to reach "hello" is 1.

Example 2:

Input: `words = ["a","b","leetcode"]`, `target = "leetcode"`, `startIndex = 0`

Output: 1

Explanation:

We start from index 0 and can reach "leetcode" by

- moving 2 units to the right to reach index 2.

- moving 1 unit to the left to reach index 2.
The shortest distance to reach "leetcode" is 1.

Example 3:

Input: words = ["i","eat","leetcode"], target = "ate", startIndex = 0

Output: -1

Explanation:

Since "ate" does not exist in words, we return -1.

Constraints:

- $1 \leq \text{words.length} \leq 100$
- $1 \leq \text{words}[i].\text{length} \leq 100$
- words[i] and target consist of only lowercase English letters.
- $0 \leq \text{startIndex} < \text{words.length}$

19. Second Largest Digit in a String

Given an alphanumeric string **s**, return the second largest numerical digit that appears in **s**, or -1 if it does not exist.

An alphanumeric string is a string consisting of lowercase English letters and digits.

Example 1:

Input: s = "dfa12321afd"

Output: 2

Explanation: The digits that appear in s are [1, 2, 3]. The second largest digit is 2.

Example 2:

Input: s = "abc1111"

Output: -1

Explanation: The digits that appear in s are [1]. There is no second largest digit.

Constraints:

- $1 \leq s.\text{length} \leq 500$
- s consists of only lowercase English letters and digits.

20. Greatest Common Divisor of Strings

For two strings **s** and **t**, we say "**t** divides **s**" if and only if **s** = **t** + **t** + **t** + ... + **t** (**t** concatenated one or more times).

Given two strings **str1** and **str2**, return the largest string **x** such that **x** divides both **str1** and **str2**.

Example 1:

Input: **str1** = "ABCABC", **str2** = "ABC"

Output: "ABC"

Example 2:

Input: **str1** = "ABABAB", **str2** = "ABAB"

Output: "AB"

Example 3:

Input: **str1** = "LEET", **str2** = "CODE"

Output: ""

Constraints:

- $1 \leq \text{str1.length}, \text{str2.length} \leq 1000$
- **str1** and **str2** consist of English uppercase letters.